RABBI ZIDNI 'ILMA

YENEPOYA

Deemed to be University

Department of Statistics

Two Days Workshop on

Programming in Python

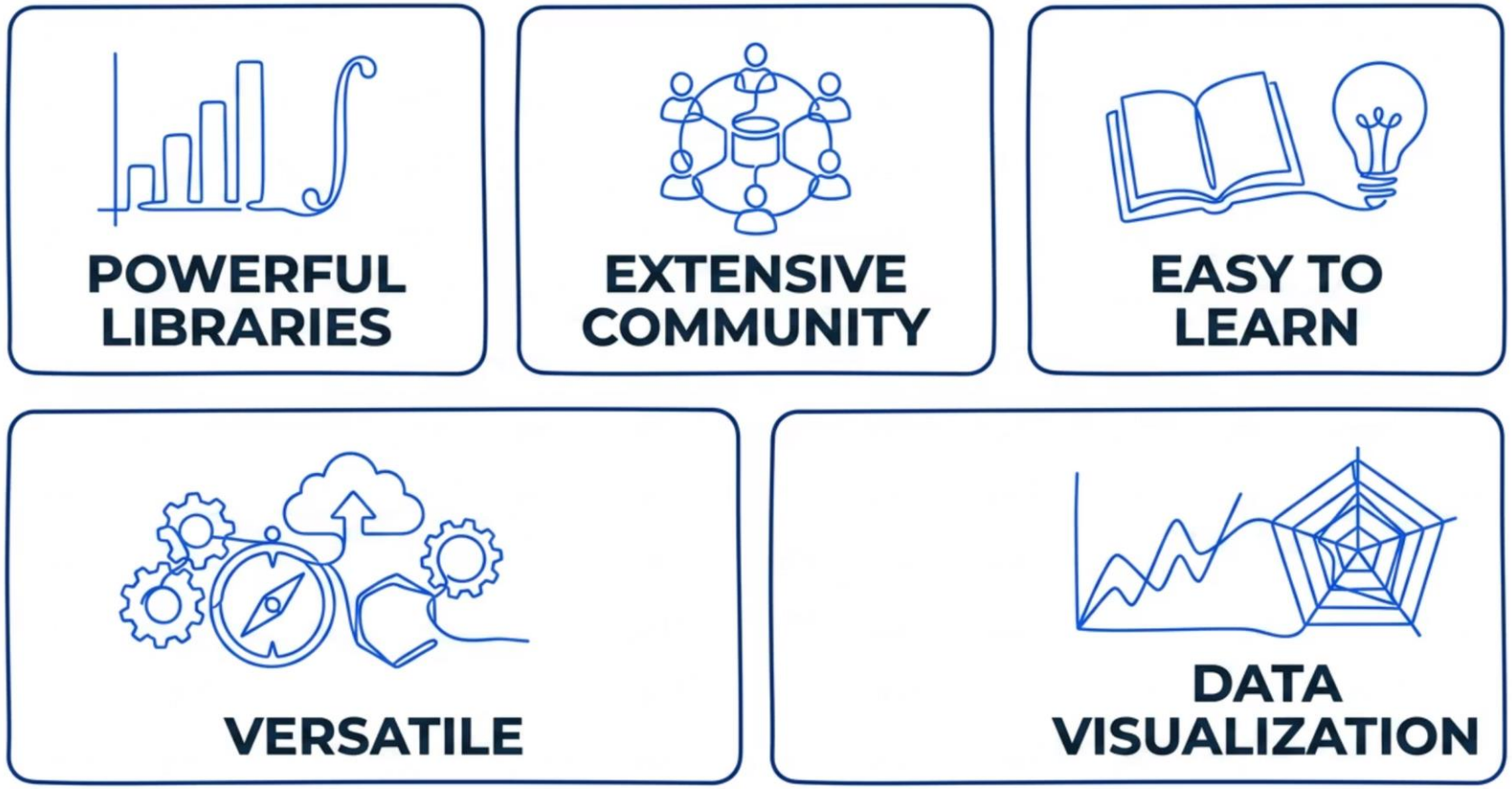
Session 1

Introduction to Python

Dr. Rathnakar Shetty P.,
Associate Professor,

rathnakar@yenepoya.edu.in
+91-9483803812

Python Dominates Data Analytics and Research



Popularity of Python

- Python consistently ranks as the world's most popular programming language — and for good reason.
- Its English-like syntax makes it accessible to beginners, while its depth and ecosystem make it indispensable for professionals.
- From building websites to training machine learning models, Python does it all with elegance.

🌍 Globally Adopted

Used by Google, NASA, Netflix, and millions of developers worldwide.

🎓 Beginner Friendly

Readable syntax that reads almost like plain English — low barrier to entry.

⚡ Versatile

Web dev, AI, automation, data science — Python spans every domain.

□ Open Source

Free to use, with a massive community contributing libraries and support.

Python is an **interpreted, high-level language** designed for readability and productivity.
Code is executed line by line, making debugging intuitive. Its clean, minimal syntax removes unnecessary complexity.

Python Playground

YENEPOYA
Deemed to be University

Anaconda

A powerful all-in-one distribution for data science — bundles Python, Jupyter, and essential libraries like NumPy and pandas out of the box.

Spider

An open-source, cross-platform Integrated Development Environment (IDE) specifically designed for scientific programming in Python

Jupyter Notebooks

An interactive environment ideal for experimentation and visualization — mix live code, outputs, and explanations in a single document.

Python Interpreter

YENEPOYA
Deemed to be University

The Python interpreter lets you execute code **line by line** and see results instantly — making it the perfect tool for learning, experimenting, and debugging. No compilation needed; simply type and run.

1

Open the Interpreter

Launch Python from your terminal by typing `python` or `python3`. You'll see the `>>>` prompt, indicating Python is ready.

2

Type an Expression

Enter any valid Python expression, such as `5 + 5`, and press Enter. Python evaluates it immediately and returns `10`.

3

Experiment Freely

Test variables, functions, and logic on the fly. The interpreter provides instant feedback, making it ideal for rapid prototyping and exploration.

ⓘ The Python interactive shell (REPL — Read, Evaluate, Print, Loop) is your best friend as a beginner. Use it to test ideas before writing full programs!

Let Us Try

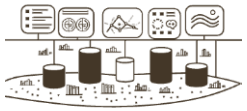
```
>>> 5 + 5
10
>>> print("Hello, Python!")
Hello, Python!
>>> 2 ** 8
256
>>> type("FDP")
<class 'str'>
```

Every line is executed immediately — no compilation needed!

Basic Building Blocks

Variables

Named containers that store data values. Variables allow you to label and reuse information throughout your program.

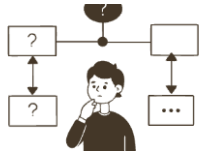


Data Types

Numbers, strings, booleans, lists, tuples, and dictionaries — Python's rich type system handles every kind of information.

Operators

Arithmetic, comparison, and logical operators let you perform calculations, make decisions, and combine conditions.



Control Flow

Loops (`for`, `while`) and conditionals (`if/else`) direct the execution path of your program dynamically.

Syntax & Data Types

YENEPOYA
Deemed to be University

Clean by Design

Python uses **indentation** instead of braces to define code blocks — enforcing readability as a core feature of the language.

```
if x > 0:
    print("Positive")
else:
    print("Non-positive")
```

int — Integers

Whole numbers: `10`, `-5`, `42`

float — Decimals

Floating-point: `3.14`, `-0.5`, `2.718`

str — Strings

Text sequences: `"Hello, Python!"`, `'World'`

bool — Booleans

Logical values: `True` or `False`

Dynamic Typing: Flexibility

In Python, you don't need to declare the type of a variable before using it. Python automatically determines the data type based on the value assigned. This is called **dynamic typing** — a feature that makes Python code faster to write and easier to read.

A variable can hold an integer one moment and a string the next. Python seamlessly adapts, allowing you to focus on logic rather than type declarations.

Example: `x = 1` makes x an integer. Then `x = "hello"` reassigns it to a string — Python handles it effortlessly with no errors!

```
x = 1           # x is an integer
print(type(x)) # <class 'int'>

x = "hello"     # x is now a string
print(type(x)) # <class 'str'>

x = 3.14        # x is now a float
print(type(x)) # <class 'float'>
```

int

Whole numbers: `x = 42`

float

Decimals: `x = 3.14`

str

Text: `x = "hello"`

bool

True/False: `x = True`

Session1_1_Identifier_Variables.ipynb

Session1_2_Operators.ipynb

Control Flow: Decisions & Repetition

YENEPOYA
Deemed to be University

Conditional Statements

Use `if`, `elif`, and `else` to execute code based on conditions — the backbone of decision-making in any program.

```
x = -5
if x < 0:
 print("Negative number")
```

```
if x > 0:
    print("Positive")
else:
    print("Non-positive")
```

```
score = 72
if score >= 90:
 print("A")
elif score >= 80:
 print("B")
elif score >= 70:
 print("C")
else:
 print("Below C")
```

Loops

Use `for` to iterate over sequences and `while` to repeat actions as long as a condition holds true.

```
# Prints: 0, 1, 2, 3, 4
for i in range(5):
    print(i)

# Start, stop, step
for i in range(2, 10, 2):
    print(i)  # 2, 4, 6, 8
```

```
count = 0
while count < 5:
 print(f"Count is {count}")
 count += 1
print("Done!")
```

Session1_3_Control Statements.ipynb

Session1_4_Looping Statements.ipynb

Repeating Actions: The `for` Loop

The `for` loop iterates over any **sequence** — lists, strings, tuples, and more. It's the ideal tool for processing every item in a collection, one by one.

```
fruits = ["apple", "banana", "cherry"]

for item in fruits:
    print(f"I like {item}")
```

Sequences

Lists, strings, tuples, sets, dicts

Predictable

Runs exactly once per item

Readable

Clear, expressive Pythonic syntax

Iterating with `range()`

The built-in `range()` function generates a sequence of integers — perfect for repeating a block of code a **specific number of times** without needing a pre-built list.

```
# Prints: 0, 1, 2, 3, 4
for i in range(5):
    print(i)

# Start, stop, step
for i in range(2, 10, 2):
    print(i)  # 2, 4, 6, 8
```

1

`range(stop)`

0 up to (not including) stop

2

`range(start, stop)`

Custom starting point

3

`range(start, stop, step)`

Custom increment between values

Data Structures: Organizing Information

YENEPOYA
Deemed to be University

Lists

Ordered, **mutable** collections.

`[1, 2, 3]`

Tuples

Ordered, **immutable** collections.

`(1, 2, 3)`

Dictionaries

Flexible **key-value** pairs.

`{'name': 'Bob'}`

Choosing the right data structure improves both the clarity and efficiency of your programs.

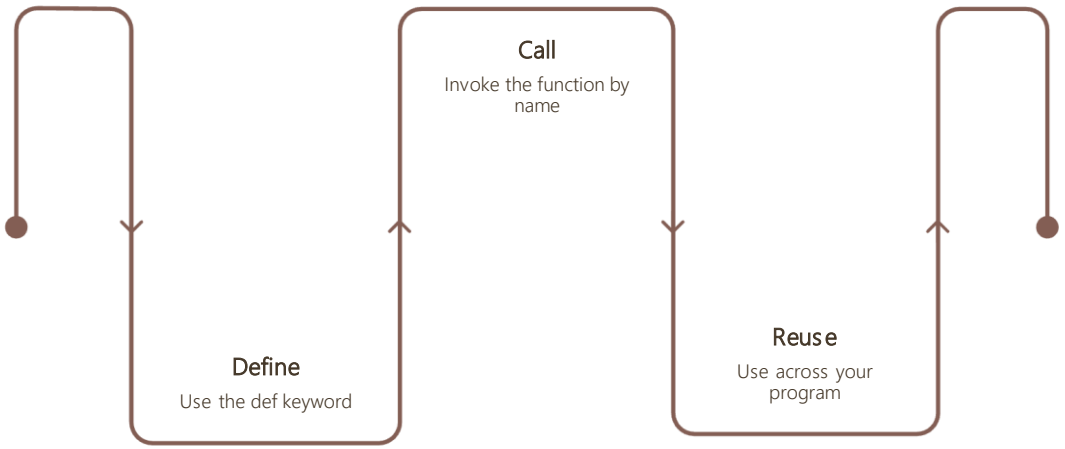
Session1_6_List.ipynb

Session1_7_Tuple.ipynb

Session1_8_Dictionaries.ipynb

Functions: Reusable Code Blocks

Functions let you encapsulate logic into named, reusable units — keeping your code **organized**, **readable**, and **DRY** (Don't Repeat Yourself).



Example

```
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
# Output: Hello, Alice!
```

Functions can accept parameters and return values, making them incredibly flexible.

Session1_10_Function.ipynb

Beyond the Basics: A Universe of Libraries

One of Python's greatest strengths is its **rich ecosystem of libraries**. These pre-built packages dramatically accelerate development, enabling you to solve complex problems with just a few lines of code.

NumPy

High-performance numerical computing with powerful N-dimensional arrays and mathematical functions.

Pandas

Intuitive data manipulation and analysis using DataFrames — the backbone of data science workflows.

Matplotlib

Create static, animated, and interactive visualizations and plots with fine-grained control.

Plotly

Build interactive, publication-quality charts and dashboards for web and notebook environments.

Scikit-learn

Industry-standard machine learning library offering classification, regression, clustering, and more.

Core Data Manipulation

- Pandas
  - The essential library for structured data manipulation. It introduces DataFrames, which allow for cleaning, filtering, and aggregating tabular data similar to Excel or SQL.
- NumPy
  - Provides support for large multi-dimensional arrays and high-level mathematical functions. It is the foundational library that many others (like Pandas and SciPy) are built upon.
- Polars
  - A newer, highly efficient alternative to Pandas designed for lightning-fast data processing on large datasets by leveraging multi-threading.

Data Visualization

- Matplotlib
  - The standard "grandfather" library for creating static, animated, and interactive 2D/3D visualizations.
- Seaborn
  - Built on top of Matplotlib, it simplifies the creation of aesthetically pleasing and complex statistical graphics like heatmaps and violin plots.
- Plotly
  - Ideal for creating highly interactive, web-ready visualizations that allow users to hover, zoom, and filter data directly.

Statistical Analysis & Scientific Computing

- SciPy
  - Extends NumPy with advanced algorithms for optimization, signal processing, integration, and linear algebra.
- Statsmodels
  - Focused on rigorous statistical modeling. It is preferred for hypothesis testing, linear regression, and time-series analysis.

Machine Learning & Advanced Analytics

- Scikit-learn
  - The go-to library for traditional machine learning tasks such as classification, regression, clustering, and dimensionality reduction.
- PyTorch & TensorFlow
  - Powerful frameworks for deep learning and building complex neural networks for tasks like image recognition and NLP.
- XGBoost & LightGBM
  - Highly optimized libraries for gradient boosting, often used for structured data competitions and high-performance predictive modeling.